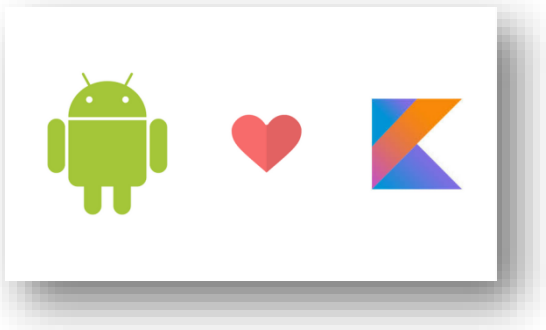
# Penyimpanan Data Eksternal Menggunakan MediaStore



## Praktikum 8

**Tujuan:**

Mahasiswa mengenal dan mengimplementasikan:

- Penyimpanan Data Eksternal Menggunakan Media Store

**Hasil Praktikum:**

- Contoh program praktikum.

**Teori MediaStore**

MediaStore adalah sebuah API yang disediakan oleh sistem operasi Android untuk mengelola dan mengakses file media seperti gambar, video, dan audio yang tersimpan di perangkat.

MediaStore merupakan bagian dari Content Provider, sehingga memungkinkan aplikasi untuk berbagi dan mengakses data media secara terstruktur tanpa harus mengakses file secara langsung melalui path penyimpanan.

Komponen Dasar MediaStore:

1. Content Resolver: Digunakan untuk berinteraksi dengan MediaStore dalam mengambil atau menyimpan data.
2. URI (Uniform Resource Identifier): Digunakan sebagai alamat untuk mengakses data media tertentu.
3. Media Collections: MediaStore mengelompokkan file menjadi beberapa kategori yaitu; Images(gambar), Video, Audio, dan Downloads.

Fungsi MediaStore:

- Mengakses file media tanpa perlu izin langsung ke file system (pada Android versi baru)
- Mengelola file media secara terstruktur

- Mendukung scoped storage (keamanan data lebih baik).

Kelebihan dari MediaStore:

- Lebih aman dan sesuai standar Android modern
- Mendukung pengelolaan file berbasis database
- Mengurangi risiko kesalahan akses file

Contoh: Mengambil daftar gambar dari penyimpanan

```
Cursor cursor = getContentResolver().query(
        MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
        null,
        null,
        null,
        null
);

if (cursor != null) {
    while (cursor.moveToNext()) {
        String imagePath = cursor.getString(
                cursor.getColumnIndex(MediaStore.Images.Media.DATA)
        );
        Log.d("IMAGE_PATH", imagePath);
    }
    cursor.close();
}
```

Penjelasan:

- MediaStore.Images.Media.EXTERNAL_CONTENT_URI → akses gambar dari storage
- Cursor → untuk membaca data hasil query
- getContentResolver() → penghubung ke MediaStore

Contoh: Menyimpan Gambar ke MediaStore

```
ContentValues values = new ContentValues();
values.put(MediaStore.Images.Media.DISPLAY_NAME, "gambar_baru.jpg");
values.put(MediaStore.Images.Media.MIME_TYPE, "image/jpeg");

Uri uri = getContentResolver().insert(
        MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
        values
);

OutputStream outputStream = getContentResolver().openOutputStream(uri);
outputStream.write(byteArrayImage);
outputStream.close();
```

Penjelasan:

- ContentValues → metadata file
- insert() → menambahkan file baru ke MediaStore
- OutputStream → menulis data gambar

**Teori Gson Library**

Gson adalah sebuah library Java yang dikembangkan oleh Google untuk mengkonversi objek Java menjadi format JSON dan sebaliknya.

Konsep Utama Gson

- Serialization : Proses mengubah objek Java menjadi JSON
- Deserialization : Proses mengubah JSON menjadi objek Java

Gson bekerja dengan cara:

- Membaca struktur objek Java
- Mengubahnya menjadi representasi JSON
- Atau sebaliknya, membaca JSON dan membangun objek Java

Contoh Penggunaan Gson

- Mengirim data ke server (API)
- Mengambil data dari web service
- Menyimpan data dalam format JSON

Kelebihan Gson

- Mudah digunakan (simple API)
- Tidak membutuhkan konfigurasi kompleks
- Mendukung nested object dan collection

Untuk menggunakan Gson tambahkan depedensi berikut pada file buil.gradle.kts (module:app):

```
implementation("com.google.code.gson:gson:2.10.1")
```

**Buat model Data Class:**

```
data class User(
    val nama: String,
    val umur: Int
)
```

**Serialization (Object → JSON)**

```
import com.google.gson.Gson

val gson = Gson()

val user = User(
    nama = "Andi",
    umur = 20
)

val json = gson.toJson(user)

println(json)
```

Output:

```
{"nama":"Andi","umur":20}
```

**Deserialization (JSON → Object)**

```
import com.google.gson.Gson

val json = """{"nama":"Budi","umur":25}"""

val gson = Gson()
val user = gson.fromJson(json, User::class.java)

println(user.nama)
println(user.umur)
```

Contoh JSON Array → List Object

```
import com.google.gson.Gson
import com.google.gson.reflect.TypeToken

val jsonArray = """
[
    {"nama":"A","umur":18},
    {"nama":"B","umur":22}
]
"""

val gson = Gson()

val type = object : TypeToken<List<User>>() {}.type
val listUser: List<User> = gson.fromJson(jsonArray, type)

listUser.forEach {
```

```
    println("${it.nama} - ${it.umur}")
}
```

Tips Penting (Biar Tidak Error)

- Nama variabel harus sama dengan key JSON
- Gunakan TypeToken untuk parsing list
- Bisa pakai @SerializedName kalau nama JSON berbeda:

```
data class User(
    @SerializedName("nama_lengkap") val nama: String,
    val umur: Int
)
```

**Teori UUID Creator Library**

UUID (Universally Unique Identifier) adalah sebuah identitas unik yang digunakan untuk membedakan suatu objek/data secara global tanpa perlu koordinasi pusat.

UUID biasanya terdiri dari 128-bit yang direpresentasikan dalam bentuk string seperti:

```
01890f47-8c3b-7cc2-b1f2-9a6f0d123456
```

Karakteristik UUID:

- Bersifat unik secara global
- Tidak mudah terjadi duplikasi
- Bisa dibuat tanpa koneksi internet
- Banyak digunakan dalam database, aplikasi, dan sistem distribusi

Versi UUID yang Umum:

1. UUID v1 → berdasarkan waktu & MAC address
2. UUID v4 → berbasis random (paling sering digunakan)
3. UUID v7 → menggabungkan keunikan global dengan urutan waktu (*time-ordered*)

Kelebihan Menggunakan UUID:

1. Tidak perlu auto-increment dari database
2. Aman untuk sistem offline
3. Cocok untuk aplikasi mobile & API
4. Menghindari konflik ID

Kekurangan UUID:

- Panjang (tidak efisien untuk dibaca manusia)
- Lebih besar ukuran penyimpanan dibanding integer

- Tidak berurutan (untuk UUID v4)

**UUID v7**

UUID versi 7 (UUIDv7) adalah versi UUID terbaru yang dirancang untuk menggabungkan keunikan global dengan urutan waktu (time-ordered) sehingga lebih optimal untuk database modern.

Berbeda dengan UUID v4 yang sepenuhnya acak, UUID v7 menggunakan:

- Timestamp (waktu saat dibuat) → membuat ID terurut
- Random bits → menjaga keunikan

Struktur UUID v7 (Sederhana)

UUID v7 terdiri dari:

- 48-bit timestamp (millisecond sejak Unix epoch)
- 74-bit random

Ciri khas:

- Angka depan berubah mengikuti waktu
- Lebih terurut dibanding UUID v4

Keunggulan UUID v7

- Sortable (berurutan berdasarkan waktu)
- Lebih optimal untuk indexing database
- Mengurangi fragmentasi index
- Tetap unik secara global

Untuk membuat UUID v7 di Kotlin, menggunakan library uuid-creator. Library ini mendukung berbagai versi UUID termasuk v7.

Menambahkan Dependency

```
implementation("com.github.f4b6a3:uuid-creator:5.3.3")
```

Contoh Pembuatan UUID v7

```
import com.github.f4b6a3.uuid.UuidCreator

fun main() {
    val uuidV7 = UuidCreator.getTimeOrderedEpoch()
    println("UUID v7: $uuidV7")
}
```

Output contoh:

```
018f1a2b-9c4d-7e11-8abc-1234567890ab
```

Contoh Penerapan UUID v7 di Kotlin ID untuk Data:

```
import com.github.f4b6a3.uuid.UuidCreator

data class User(
    val id: String,
    val nama: String
)

fun main() {
    val user = User(
        id = UuidCreator.getTimeOrderedEpoch().toString(),
        nama = "Andi"
    )

    println(user)
}
```

Keunggulan: ID terurut berdasarkan waktu pembuatan.

UUID v7 untuk Database:

```
import com.github.f4b6a3.uuid.UuidCreator

fun generateId(): String {
    return UuidCreator.getTimeOrderedEpoch().toString()
}
```

Cocok untuk:

- Primary key database
- Log data
- Sistem transaksi

UUID v7 untuk File:

```
import com.github.f4b6a3.uuid.UuidCreator

fun main() {
    val fileName = "${UuidCreator.getTimeOrderedEpoch()}.jpg"
    println(fileName)
}
```

Keuntungan:

- Nama file otomatis unik
- Bisa diurutkan berdasarkan waktu upload

Sorting UUID v7:

```
import com.github.f4b6a3.uuid.UuidCreator

fun main() {
    val list = List(5) {
        UuidCreator.getTimeOrderedEpoch().toString()
    }

    list.sorted().forEach {
        println(it)
    }
}
```

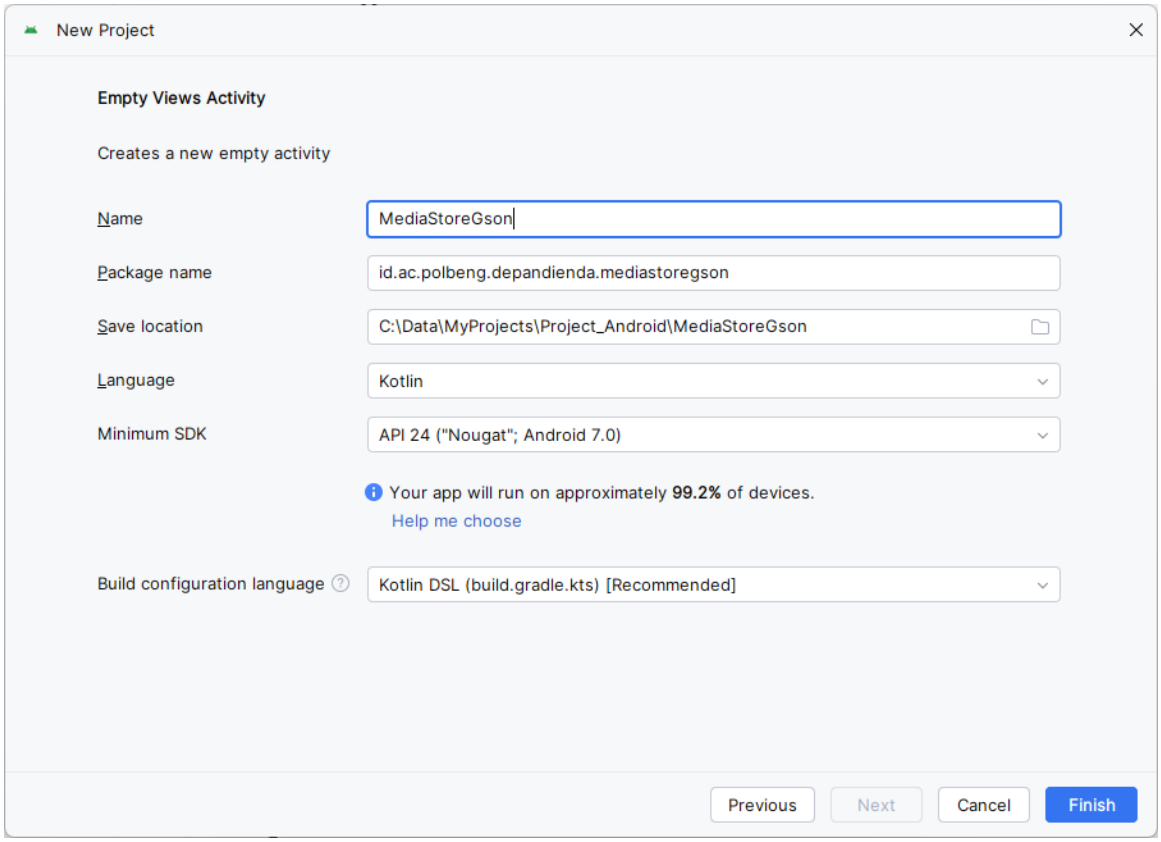
Karena berbasis waktu → hasil sorting sesuai urutan pembuatan.

Pada latihan ini kita akan menggunakan UUID v7 dalam mengenerate ID dari user.

**Latihan Mengimplementasikan MediaStore, Gson dan UUID Library**

Langkah-langkah penerapan MediaStore, Gson, dan UUID Library adalah sebagai berikut:

1. Buatlah project baru dengan nama “MediaStoreGson”.



2. Jangan lupa untuk mengaktifkan view-binding dengan menambahkan kode berikut pada build.gradle di level module build.gradle(Module: app), dan tambahkan juga dependency atau library ***loopj.***

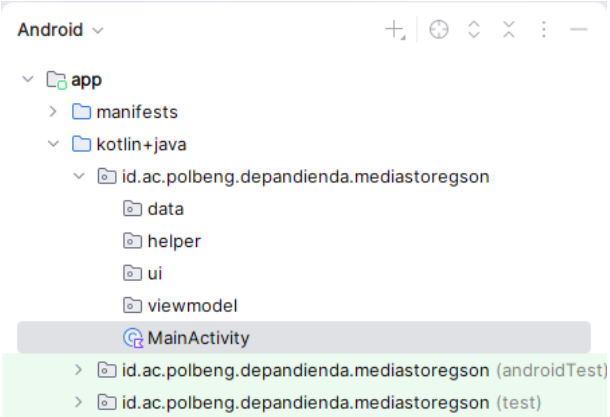
```
android {
    ...
    buildFeatures {
        viewBinding = true
    }
}
dependencies {
    ...
    implementation(libs.gson)
    implementation(libs.uuid.creator)
}
```

Tambahkan juga deklarasi versi library yang digunakan di file libs.versions.toml.

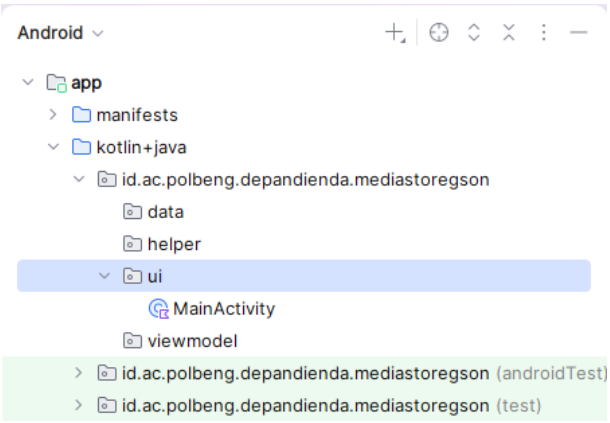
```
[versions]
...
gson = "2.14.0"
uuidCreator = "6.1.1"
...
[libraries]
...
gson = { module = "com.google.code.gson:gson", version.ref = "gson" }
uuid-creator = { module = "com.github.f4b6a3:uuid-creator", version.ref =
"uuidCreator" }
...
```

Jangan lupa klik teks **sync now** untuk menerapkan perubahan library.

3. Buatlah package baru dengan nama data, viewmodel, helpers dan ui di dalam package utama aplikasi dengan cara klik kanan paket utama New → Package. Sehingga menjadi seperti berikut:




Pindahkan file MainActivity ke dalam paket ui.




4. Selanjutnya editlah file strings.xml pada folder values, tambahkan beberapa variabel dan nilai string sebagai berikut:

```
<resources>
    <string name="app_name">MediaStoreGson</string>
    <string name="title">User Data</string>
    <string name="id">ID : Auto Filled</string>
    <string name="nama">Nama</string>
    <string name="email">Email</string>
    <string name="add">Add</string>
    <string name="update">Update</string>
    <string name="export">Export</string>
    <string name="imports">Import</string>
    <string name="edit">Edit</string>
    <string name="delete">Delete</string>
</resources>
```

5. Tambahkan defenisi warna baru red dan green di dalam file colors.xml pada folder values. Masukan kode berikut dalam file colors.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="red">#FF0000</color>
    <color name="green">#28a745</color>
</resources>
```

6. Selanjutnya tambahkan internet permission di file AndroidManifest.xml.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission
android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

```
    <application
        ...
    </application>

</manifest>
```

7. Buatlah layout baru dengan nama user_item dalam folder layout, masukan kode berikut:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:padding="10dp"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <TextView
        android:id="@+id/tvId"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        tools:text="ID"/>

    <TextView
        android:id="@+id/tvName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        tools:text="Name"/>

    <TextView
        android:id="@+id/tvEmail"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        tools:text="Email"/>

    <LinearLayout
        android:orientation="horizontal"
        android:layout_marginTop="8dp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content">

        <Button
            android:id="@+id/btnEdit"
            android:text="@string/edit"
            android:backgroundTint="@color/green"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginEnd="5dp"/>
```

```
        <Button
            android:id="@+id/btnDelete"
            android:text="@string/delete"
            android:backgroundTint="@color/red"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content" />
    </LinearLayout>

</LinearLayout>
```

8. Editlah kode pada file **activity_main.xml,** masukan kode berikut:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    tools:context=".ui.MainActivity">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/title"
        android:textSize="25sp"
        android:textStyle="bold"
        android:layout_gravity="center_horizontal"
        android:layout_marginTop="16dp"/>

    <EditText
        android:id="@+id/etId"
        android:hint="@string/id"
        android:enabled="false"
        android:inputType="number"
        android:layout_width="match_parent"
        android:layout_height="48dp"
        android:layout_marginTop="16dp"/>

    <EditText
        android:id="@+id/etName"
        android:hint="@string/nama"
        android:layout_width="match_parent"
        android:layout_height="48dp"/>

    <EditText
        android:id="@+id/etEmail"
```

```
        android:hint="@string/email"
        android:inputType="textEmailAddress"
        android:layout_width="match_parent"
        android:layout_height="48dp"/>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:weightSum="1.2"
        android:layout_marginTop="10dp">

        <Button
            android:id="@+id/btnAdd"
            android:text="@string/add"
            android:layout_weight="0.4"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginEnd="5dp"/>

        <Button
            android:id="@+id/btnExport"
            android:text="@string/export"
            android:layout_weight="0.4"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginEnd="5dp"/>

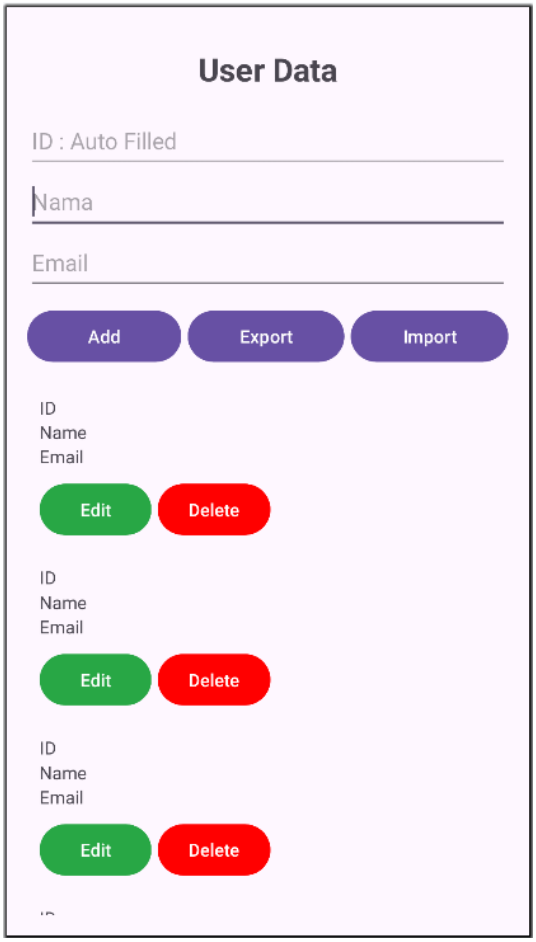
        <Button
            android:id="@+id/btnImport"
            android:text="@string/imports"
            android:layout_weight="0.4"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>

    </LinearLayout>

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerView"
        android:layout_marginTop="12dp"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        tools:listitem="@layout/user_item"/>

</LinearLayout>
```

Hasil antarmuka activity_main.xml adalah seperti berikut:




9. Buatlah kelas model baru dengan nama User dan masukan kedalam paket data. Caranya klik kanan paket data → New → Kotlin Class/File, masukan kode berikut:

```
package id.ac.polbeng.depandienda.mediastoregson.data

import com.github.f4b6a3.uuid.UuidCreator

data class User (
    val id: String = UuidCreator.getTimeOrdered().toString(),
    val name: String,
    val email: String
)
```

10. Buatlah kelas kofigurasi baru dengan nama Config dan masukan kedalam paket helper. Caranya klik kanan paket helper → New → Kotlin Class/File, masukan kode berikut:

```
package id.ac.polbeng.depandienda.mediastoregson.helper

import android.os.Environment
```

```
class Config {
    companion object{
        const val FILE_NAME = "user.json"
        val RELATIVE_PATH = "${Environment.DIRECTORY_DOCUMENTS}/MyApp/"
        const val MIME_TYPE = "application/json"
        const val VOLUME_NAME = "external"
    }
}
```

11. Buat kelas repositori baru dengan nama UserRepository di dalam paket data. Caranya klik kanan pada paket data → New → Kotlin Class/File, masukan kode berikut:

```
package id.ac.polbeng.depandienda.mediastoregson.data

import android.content.ContentValues
import android.content.Context
import android.net.Uri
import android.provider.MediaStore
import com.google.gson.Gson
import com.google.gson.reflect.TypeToken
import id.ac.polbeng.depandienda.mediastoregson.helper.Config
import java.io.BufferedReader
import java.io.InputStreamReader

class UserRepository(private val context: Context) {

    fun saveUsers(users: List<User>) {
        val json = Gson().toJson(users)
        val resolver = context.contentResolver

        val contentValues = ContentValues().apply {
            put(MediaStore.MediaColumns.DISPLAY_NAME, Config.FILE_NAME)
            put(MediaStore.MediaColumns.MIME_TYPE, Config.MIME_TYPE)
            put(MediaStore.MediaColumns.RELATIVE_PATH,
Config.RELATIVE_PATH)
        }

        val uri = getFileUri() ?: resolver.insert(
            MediaStore.Files.getContentUri(Config.VOLUME_NAME),
            contentValues
        )

        uri?.let {
            try {
                resolver.openOutputStream(it, "rwt")?.use { outputStream
->
                    outputStream.write(json.toByteArray())
                }
```

```
            } catch (e: Exception) {
                e.printStackTrace()
            }
        }
    }

    fun loadUsers(): MutableList<User> {
        val uri = getFileUri() ?: return mutableListOf()

        return try {
            val inputStream =
context.contentResolver.openInputStream(uri)
            val reader = BufferedReader(InputStreamReader(inputStream))

            val json = reader.readText()
            val type = object : TypeToken<MutableList<User>>() {}.type

            Gson().fromJson(json, type)
        } catch (e: Exception) {
            mutableListOf()
        }
    }

    private fun getFileUri(): Uri? {
        val resolver = context.contentResolver

        val projection = arrayOf(MediaStore.MediaColumns._ID)
        val selection = "${MediaStore.MediaColumns.DISPLAY_NAME}=? AND
${MediaStore.MediaColumns.RELATIVE_PATH}=?"
        val selectionArgs = arrayOf(Config.FILE_NAME,
Config.RELATIVE_PATH)

        val queryUri = MediaStore.Files.getContentUri(Config.VOLUME_NAME)

        resolver.query(queryUri, projection, selection, selectionArgs,
null)
            ?.use { cursor ->
                if (cursor.moveToFirst()) {
                    val id =
cursor.getLong(cursor.getColumnIndexOrThrow(MediaStore.MediaColumns._ID))
                    return Uri.withAppendedPath(queryUri, id.toString())
                }
            }

        return null
    }

    fun exportToUri(uri: Uri, users: List<User>) {
        val json = Gson().toJson(users)
```

```
        try {
            context.contentResolver.openOutputStream(uri)?.use {
                it.write(json.toByteArray())
            }
        } catch (e: Exception) {
            e.printStackTrace()
        }
    }

    fun importFromUri(uri: Uri): MutableList<User> {
        return try {
            val inputStream =
context.contentResolver.openInputStream(uri)
            val json = inputStream?.bufferedReader().use { it?.readText()
}

            val type = object : TypeToken<MutableList<User>>() {}.type
            Gson().fromJson(json, type)
        } catch (_: Exception) {
            mutableListOf()
        }
    }
}
```

12. Setelah membuat repository tahap selanjutnya membuat adapter untuk RecyclerView. Buatlah kelas baru dengan nama UserAdapter di dalam paket ui. Caranya klik kanan pada paket ui → New → Kotlin Class/File, masukan kode berikut:

```
package id.ac.polbeng.depandienda.mediastoregson.ui

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.DiffUtil
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import id.ac.polbeng.depandienda.mediastoregson.data.User
import
id.ac.polbeng.depandienda.mediastoregson.databinding.UserItemBinding

class UserAdapter(
    private val onEdit: (User) -> Unit,
    private val onDelete: (User) -> Unit
) :
    ListAdapter<User, UserAdapter.ViewHolder>(DIFF_CALLBACK) {

    companion object {
        val DIFF_CALLBACK = object : DiffUtil.ItemCallback<User>() {
            override fun areItemsTheSame(oldItem: User, newItem: User):
```

```
Boolean {
                return oldItem.id == newItem.id
            }

            override fun areContentsTheSame(oldItem: User, newItem:
User): Boolean {
                return oldItem == newItem
            }
        }
    }

    class ViewHolder(val binding: UserItemBinding):
RecyclerView.ViewHolder(binding.root)

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
ViewHolder {
        val binding =
UserItemBinding.inflate(LayoutInflater.from(parent.context), parent,
false)
        return ViewHolder(binding)
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        val user = getItem(position)
        holder.binding.tvId.text = "ID: ${user.id}"
        holder.binding.tvName.text = "Nama: ${user.name}"
        holder.binding.tvEmail.text = "Email: ${user.email}"

        holder.binding.btnEdit.setOnClickListener {
            onEdit(user)
        }

        holder.binding.btnDelete.setOnClickListener {
            onDelete(user)
        }
    }
}
```

13. Langkah berikutnya adalah membuat UserViewModel dalam paket viewmodel. Caranya klik kanan pada paket viewmodel → New → Kotlin Class/File, masukan kode berikut:

```
package id.ac.polbeng.depandienda.mediastoregson.viewmodel

import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel
import android.net.Uri
import id.ac.polbeng.depandienda.mediastoregson.data.User
import id.ac.polbeng.depandienda.mediastoregson.data.UserRepository
```

```
class UserViewModel(private val repository: UserRepository) : ViewModel()
{

    private val _users = MutableLiveData<List<User>>(emptyList())
    private var currentList = mutableListOf<User>()
    private var deletedUser: User? = null

    private fun emit() {
        _users.value = currentList.sortedByDescending { it.id }
        repository.saveUsers(currentList)
    }

    fun loadUsers() {
        currentList.clear()
        currentList.addAll(repository.loadUsers())
        _users.value = currentList.sortedByDescending { it.id }
    }

    fun addOrUpdate(user: User) {
        val index = currentList.indexOfFirst { it.id == user.id }

        if (index >= 0) {
            currentList[index] = user
        } else {
            currentList.add(user)
        }

        emit()
    }

    fun deleteUser(user: User) {
        val index = currentList.indexOfFirst { it.id == user.id }
        if (index == -1) return

        deletedUser = user
        currentList.removeAt(index)

        println(currentList.toString())
        println(index)

        emit()
    }

    fun undoDelete() {
        deletedUser?.let { user ->
            if (currentList.none { it.id == user.id }) {
                currentList.add(user)
                emit()
```

```
            }
        }
        deletedUser = null

        emit()
    }

    fun exportToUri(uri: Uri) {
        repository.exportToUri(uri, currentList)
    }

    fun importFromUri(uri: Uri) {
        val imported = repository.importFromUri(uri)

        currentList.clear()
        currentList.addAll(imported)

        emit()
    }

    fun mergeImport(uri: Uri) {
        val importedList = repository.importFromUri(uri)
        if (importedList.isEmpty()) return

        val existingMap = currentList.associateBy { it.id
}.toMutableMap()

        for (user in importedList) {
            if (!existingMap.containsKey(user.id)) {
                existingMap[user.id] = user
            }
        }

        currentList.clear()
        currentList.addAll(existingMap.values)

        emit()
    }
}
```

Tambahkan juga kelas UserViewModelFactory pada paket viewmodel. Caranya klik kanan pada paket viewmodel → New → Kotlin Class/File, masukan kode berikut:

```
package id.ac.polbeng.depandienda.mediastoregson.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import id.ac.polbeng.depandienda.mediastoregson.data.UserRepository
```

```
class UserViewModelFactory(private val repository: UserRepository) :
ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        return UserViewModel(repository) as T
    }
}
```

14. Terakhir editlah kode pada file ActivityMain.kt menjadi seperti berikut.

```
package id.ac.polbeng.depandienda.webapiexample

import android.os.Bundle
import android.util.Log
import android.view.View
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.loopj.android.http.AsyncHttpClient
import com.loopj.android.http.AsyncHttpResponseHandler
import cz.msebera.android.httpclient.Header
import
id.ac.polbeng.depandienda.webapiexample.databinding.ActivityMainBinding
import org.json.JSONObject

class MainActivity : AppCompatActivity() {

    companion object {
        private val TAG = MainActivity::class.java.simpleName
    }

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        getRandomQuote()
    }

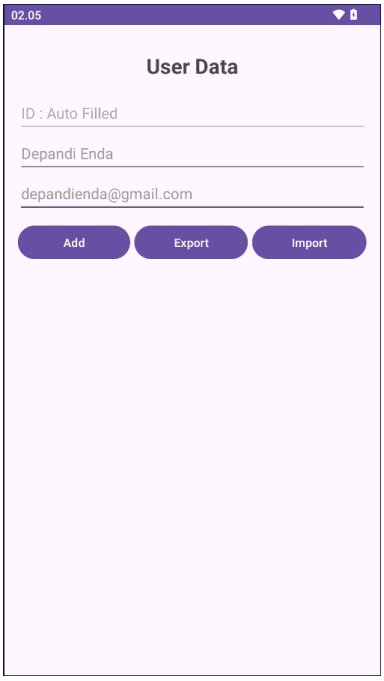
    private fun getRandomQuote() {
        binding.progressBar.visibility = View.VISIBLE
        val client = AsyncHttpClient()
        val url = "https://quote-api.dicoding.dev/random"
        client.get(url, object : AsyncHttpResponseHandler() {
            override fun onSuccess(statusCode: Int, headers:
Array<Header>, responseBody: ByteArray) {
                // Jika koneksi berhasil
                binding.progressBar.visibility = View.INVISIBLE
```

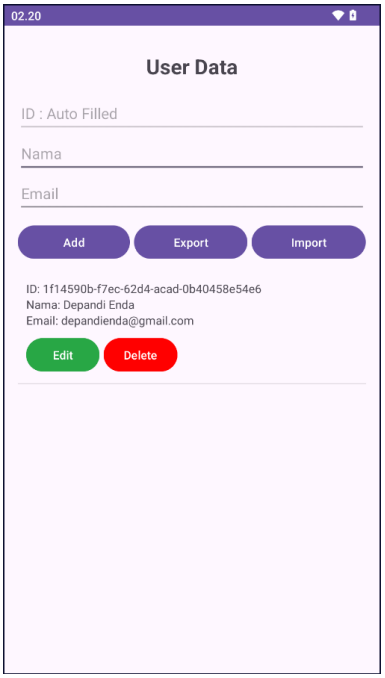
```
                val result = String(responseBody)
                Log.d(TAG, result)
                try {
                    val responseObject = JSONObject(result)
                    val quote = responseObject.getString("en")
                    val author = responseObject.getString("author")
                    binding.tvQuote.text = quote
                    binding.tvAuthor.text = author
                } catch (e: Exception) {
                    Toast.makeText(this@MainActivity, e.message,
Toast.LENGTH_SHORT).show()
                    e.printStackTrace()
                }

            }

            override fun onFailure(statusCode: Int, headers:
Array<Header>, responseBody: ByteArray, error: Throwable) {
                // Jika koneksi gagal
                binding.progressBar.visibility = View.INVISIBLE
                val errorMessage = when (statusCode) {
                    401 -> "$statusCode : Bad Request"
                    403 -> "$statusCode : Forbidden"
                    404 -> "$statusCode : Not Found"
                    else -> "$statusCode : ${error.message}"
                }
                Toast.makeText(this@MainActivity, errorMessage,
Toast.LENGTH_SHORT).show()
            }
        })
    }
}
```

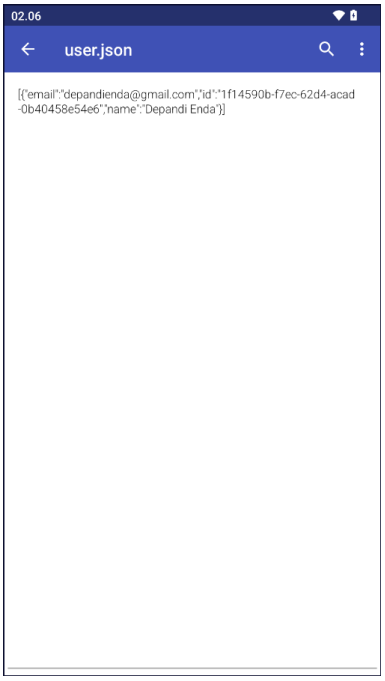
Jalankan program, coba tambahkan data Nama dan Email maka hasilnya adalah seperti berikut:



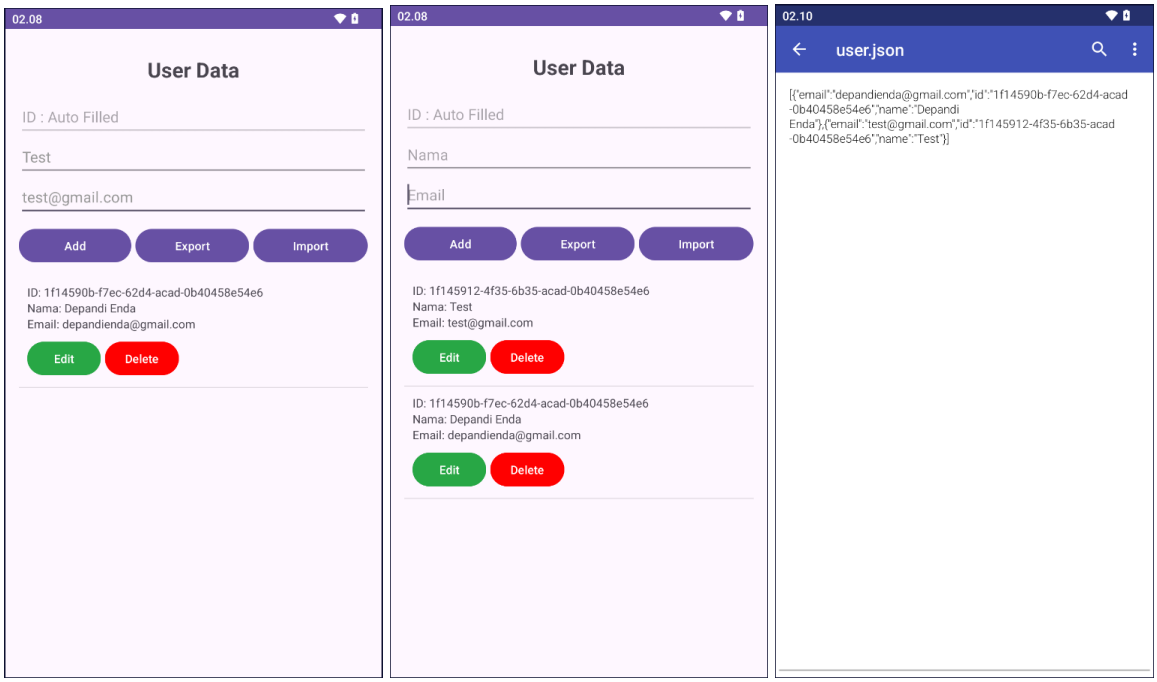




Coba cek file user.json di dalam folder Documents perangkat Anda maka hasilnya akan terbentuk folder baru dengan nama MyApp dimana ada sebuah file dengan nama user.json seperti berikut:

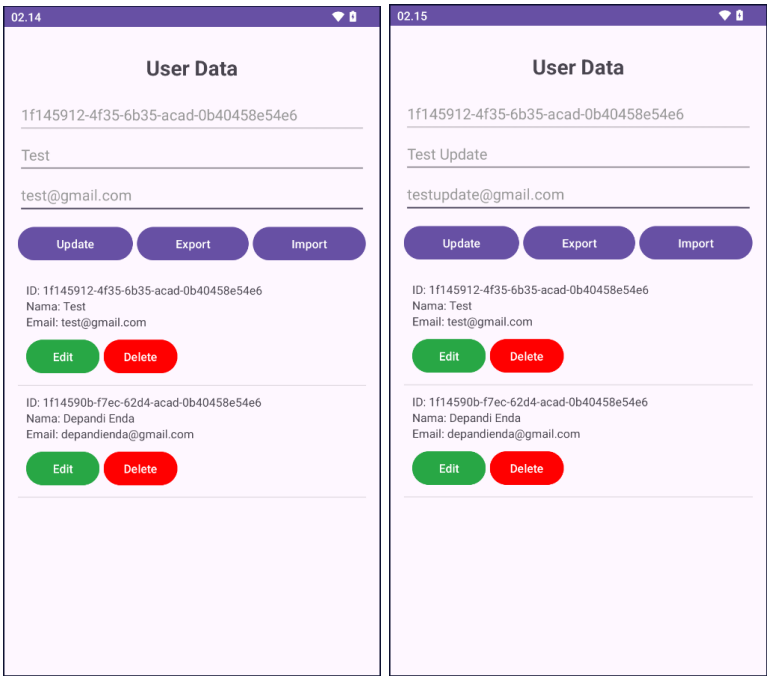


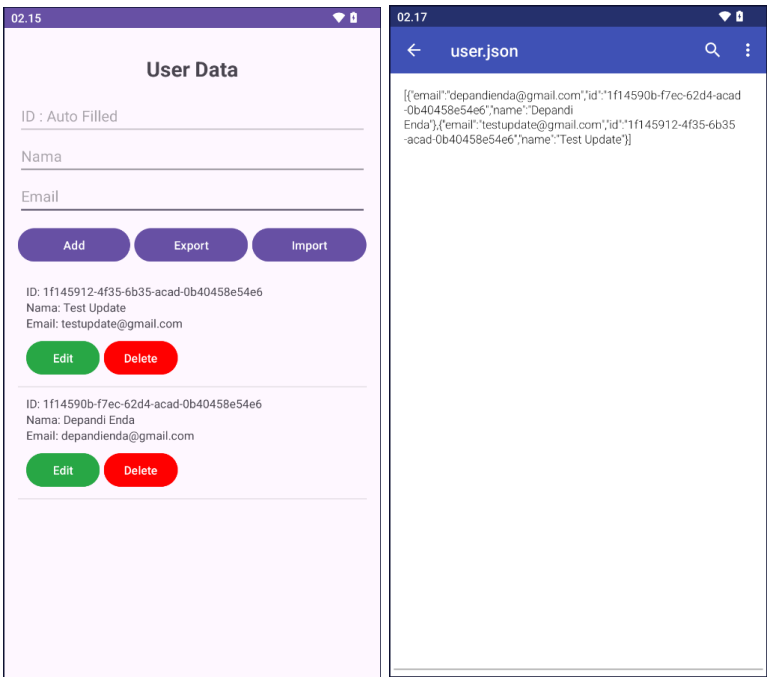

Coba tambahkan lagi data baru dan cek juga perubahan datanya di dalam file user.json seperti berikut:



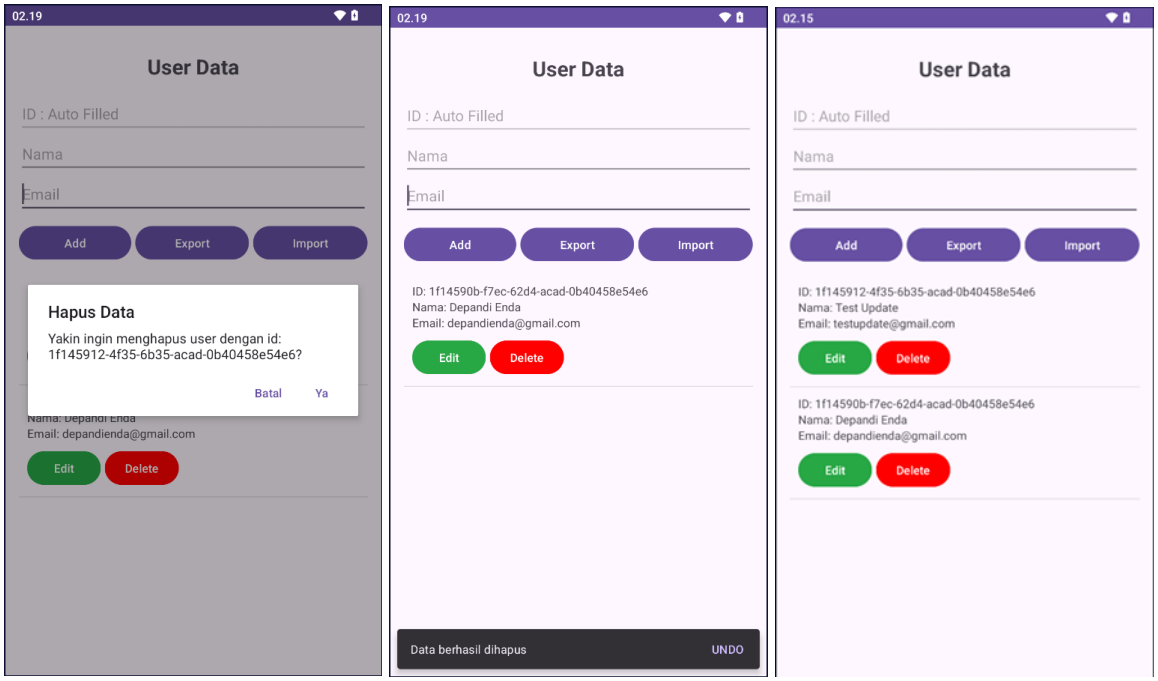

Cobalah untuk mengupdate data Test menjadi Test Update dengan email menjadi testupdate@gmail.com dengan mengklik button edit pada data Test tersebut, maka data akan ditampilkan pada EditText nama dan email secara langsung seperti berikut:



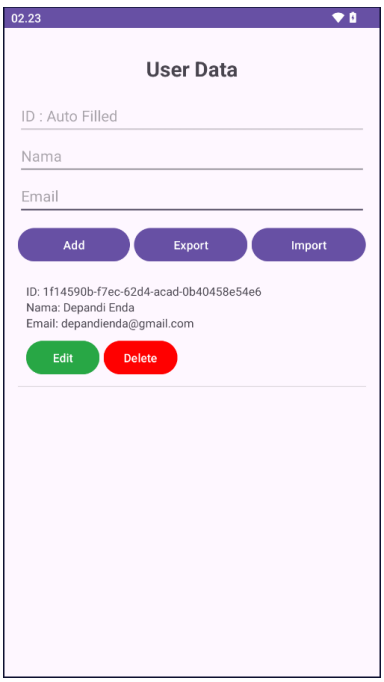

Coba klik button Update dan perhatikan data pada list dan juga pada file user.json. Maka hasilnya seharusnya seperti berikut:

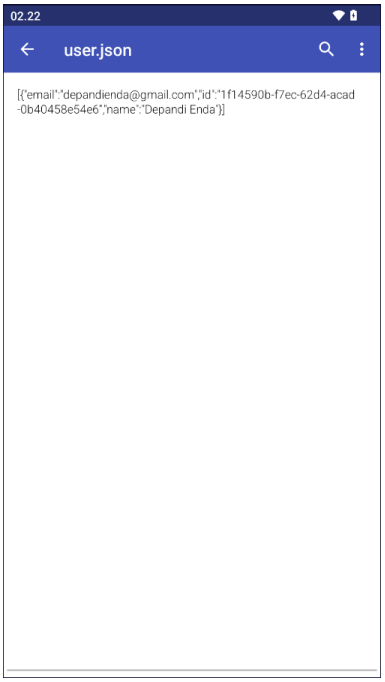



Sekarang cobalah untuk menghapus data Test Update, dan langsung klik undo ketika tampil snackbar Undo, seharusnya data tidak akan terhapus. Coba cek juga di file user.json untuk memastikan.

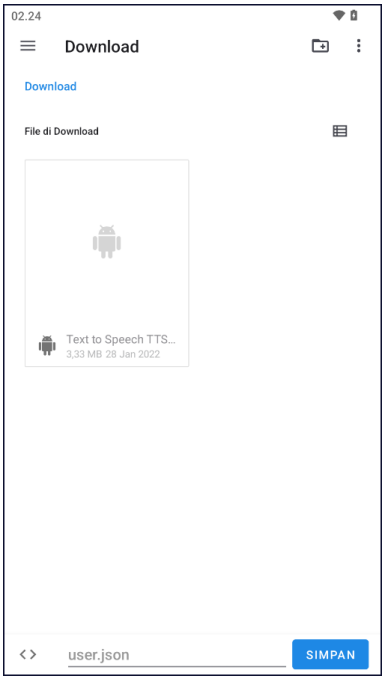


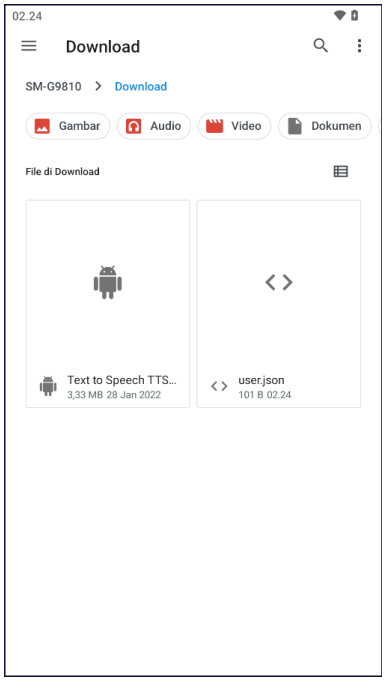

Sekarang coba hapus lagi secara permanen tanpa mengklik UNDO maka data Test Update seharusnya sudah tidak tampil lagi di list dan juga terhapus di dalam file user.json.

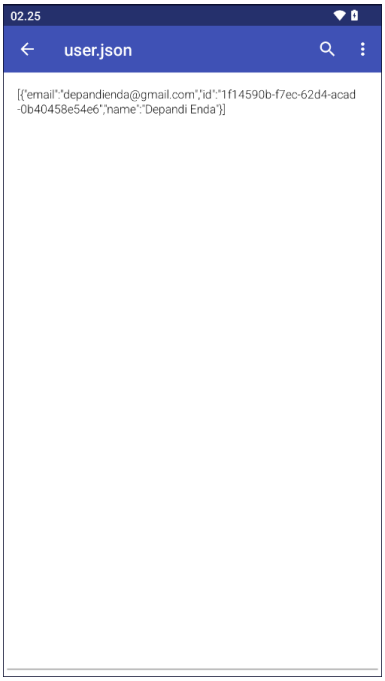


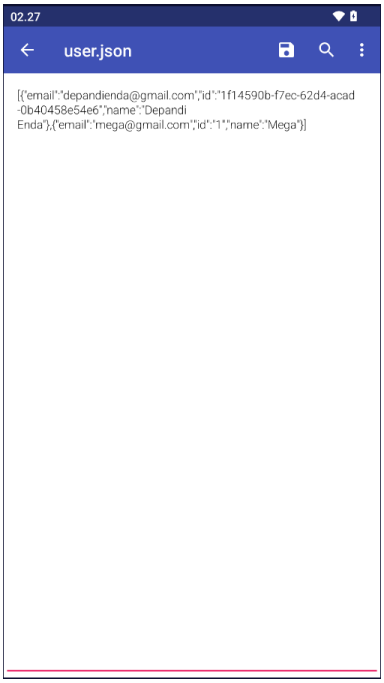

Selanjutnya cobalah untuk mengekspor data. Hasilnya adalah seperti berikut:

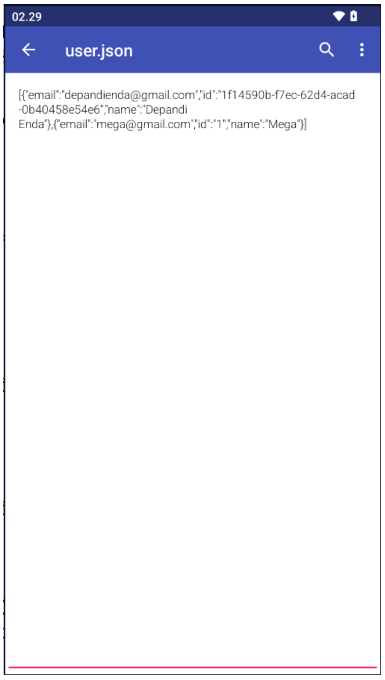


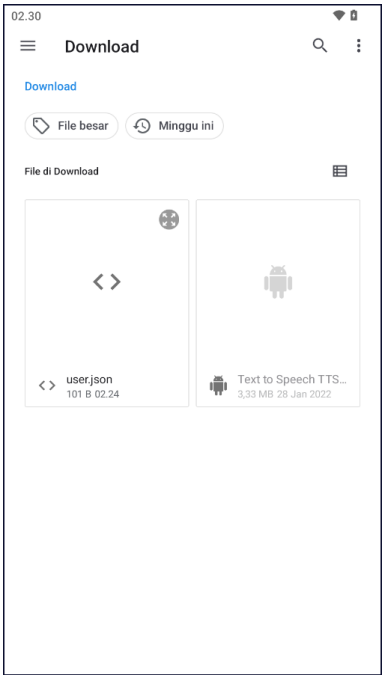



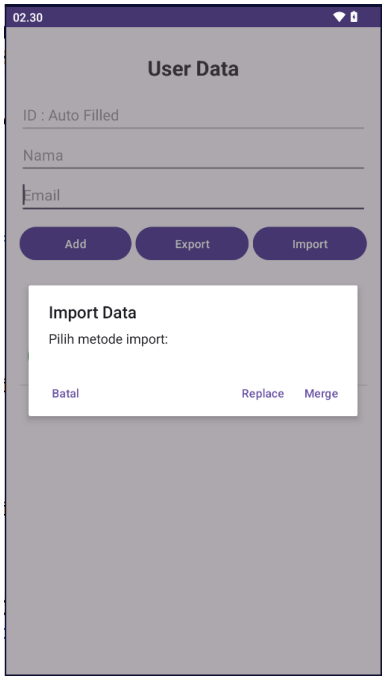

Coba juga fitur import. Ubahlah data user.json di dalam folder Download menjadi seperti berikut (jangan lupa untuk menekan tombol save untuk menyimpan perubahan data):

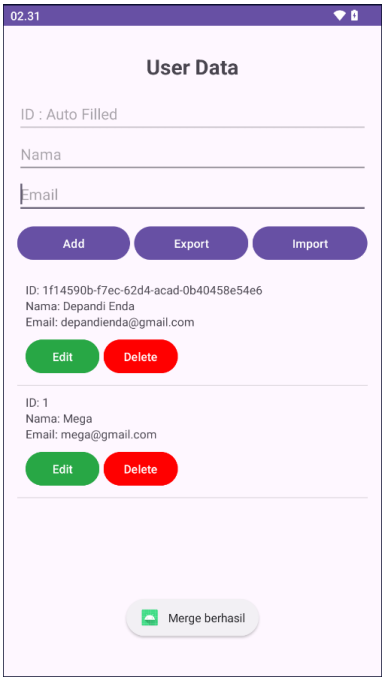


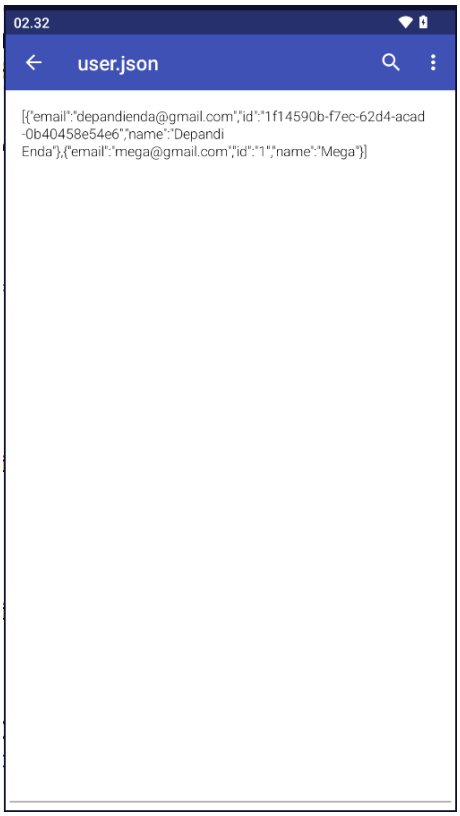

Setelah diubah cobalah lakukan import data, dari file user.json dalam folder Download ini dengan mode Merge. Jika tidak error maka hasilnya seperti berikut:




Coba cek kembali data pada file user.json di folder MyApp. Hasilnya seharusnya seperti berikut:




Jika data yang di-import ada. Maka Selamat! Anda telah berhasil menerapkan penyimpanan data eksternal menggunakan MediaStore, Gson dan UUIDCreator Library dengan benar.

**Tugas Praktikum**

- Tuliskan pemahaman anda mengenai contoh koding program praktikum 8 dan screen shoot hasilnya.
- Buatlah laporan resmi praktikum 8.

**Daftar Referensi**

https://www.dicoding.com/

https://developer.android.com

** ☺ Selamat Mengerjakan ☺ ***